```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session
```

Imports + Loader (CMAPSS)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (12, 5)

def load_cmapss_split(data_root, fd="FD001"):
    train_path = f"{data_root}/train_{fd}.txt"
    test_path  = f"{data_root}/test_{fd}.txt"
    rul_path   = f"{data_root}/RUL_{fd}.txt"

    train = pd.read_csv(train_path, sep=r"\s+", header=None, engine="python")
    test  = pd.read_csv(test_path,  sep=r"\s+", header=None, engine="python")
    rul   = pd.read_csv(rul_path,   sep=r"\s+", header=None, engine="python")

    cols = ["unit_id", "cycle"] + [f"op_setting_{i}" for i in range(1, 4)] + [f"s{i}" for i in range(1, 22)]
    train.columns = cols
    test.columns  = cols
    rul.columns   = ["RUL_true"]

    return train, test, rul

def add_train_rul(train_df):
    df = train_df.copy()
    df["RUL"] = df.groupby("unit_id")["cycle"].transform("max") - df["cycle"]
    return df

def add_test_rul(test_df, rul_df):
    df = test_df.copy()
    last_cycle = df.groupby("unit_id")["cycle"].max().rename("last_cycle")
    tmp = last_cycle.to_frame().join(rul_df.reset_index(drop=True))
    tmp["max_cycle_est"] = tmp["last_cycle"] + tmp["RUL_true"].values
    df = df.join(tmp["max_cycle_est"], on="unit_id")
    df["RUL"] = df["max_cycle_est"] - df["cycle"]
    df.drop(columns=["max_cycle_est"], inplace=True)
    return df
```

```
import os
print(os.listdir("/content"))
```

```
['.config', 'test_FD001.txt', 'RUL_FD001.txt', 'train_FD001.txt', 'sample_data']
```

```
DATA_ROOT = "/content"
```

Load Dataset

```
FD = "FD001"
train_raw, test_raw, rul_true = load_cmapss_split(DATA_ROOT, FD)
print(train_raw.shape, test_raw.shape, rul_true.shape)
train_raw.head()
```

```
(20631, 26) (13096, 26) (100, 1)
```

	unit_id	cycle	op_setting_1	op_setting_2	op_setting_3	s1	s2	s3	s4	s5	...	s12	s13	s14
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60	14.62	...	521.66	2388.02	8138.62
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14	14.62	...	522.28	2388.07	8131.49
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1404.20	14.62	...	522.42	2388.03	8133.23
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1401.87	14.62	...	522.86	2388.08	8133.83
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1406.22	14.62	...	522.19	2388.04	8133.80

5 rows × 26 columns

Quick Data Audit (Quality + Types + Missing)

```
FD = "FD001"

train_raw, test_raw, rul_true = load_cmapss_split(DATA_ROOT, FD)
train = add_train_rul(train_raw)
test  = add_test_rul(test_raw, rul_true)

print(train.shape, test.shape)
```

```
(20631, 27) (13096, 27)
```

```
def audit(df, name="df"):
    print(f"\n### {name}")
    display(df.head(5))
    print("\ninfo:")
    df.info()
    print("\nmissing (top 10):")
    display(df.isna().sum().sort_values(ascending=False).head(10))
    print("\nduplicates:", df.duplicated().sum())

audit(train, "Train (with RUL)")
audit(test,  "Test (with RUL)")
```

```
### Train (with RUL)
```

	unit_id	cycle	op_setting_1	op_setting_2	op_setting_3	s1	s2	s3	s4	s5	...	s13	s14	s15
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60	14.62	...	2388.02	8138.62	8.4195
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14	14.62	...	2388.07	8131.49	8.4318
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1404.20	14.62	...	2388.03	8133.23	8.4178
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1401.87	14.62	...	2388.08	8133.83	8.3682
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1406.22	14.62	...	2388.04	8133.80	8.4294

5 rows × 27 columns

```
info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20631 entries, 0 to 20630
Data columns (total 27 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   unit_id       20631 non-null  int64
 1   cycle         20631 non-null  int64
 2   op_setting_1  20631 non-null  float64
 3   op_setting_2  20631 non-null  float64
 4   op_setting_3  20631 non-null  float64
 5   s1            20631 non-null  float64
 6   s2            20631 non-null  float64
 7   s3            20631 non-null  float64
 8   s4            20631 non-null  float64
 9   s5            20631 non-null  float64
 10  s6            20631 non-null  float64
 11  s7            20631 non-null  float64
 12  s8            20631 non-null  float64
 13  s9            20631 non-null  float64
 14  s10           20631 non-null  float64
 15  s11           20631 non-null  float64
 16  s12           20631 non-null  float64
 17  s13           20631 non-null  float64
 18  s14           20631 non-null  float64
 19  s15           20631 non-null  float64
 20  s16           20631 non-null  float64
 21  s17           20631 non-null  int64
 22  s18           20631 non-null  int64
 23  s19           20631 non-null  float64
 24  s20           20631 non-null  float64
 25  s21           20631 non-null  float64
 26  RUL           20631 non-null  int64
dtypes: float64(22), int64(5)
memory usage: 4.2 MB

missing (top 10):
```

	0
unit_id	0
cycle	0
op_setting_1	0
op_setting_2	0
op_setting_3	0
s1	0
s2	0
s3	0
s4	0
s5	0

dtype: int64

3) Engine-level Summary (lifetimes + rows)

```
train_eng = train.groupby("unit_id").agg(
    max_cycle=("cycle","max"),
    min_cycle=("cycle","min"),
    n_rows=("cycle","count"),
    start_rul=("RUL","max"),
    end_rul=("RUL","min"),
).reset_index()
```

```
test_eng = test.groupby("unit_id").agg(
    max_cycle=("cycle","max"),
    min_cycle=("cycle","min"),
    n_rows=("cycle","count"),
    start_rul=("RUL","max"),
    end_rul=("RUL","min"),
).reset_index()

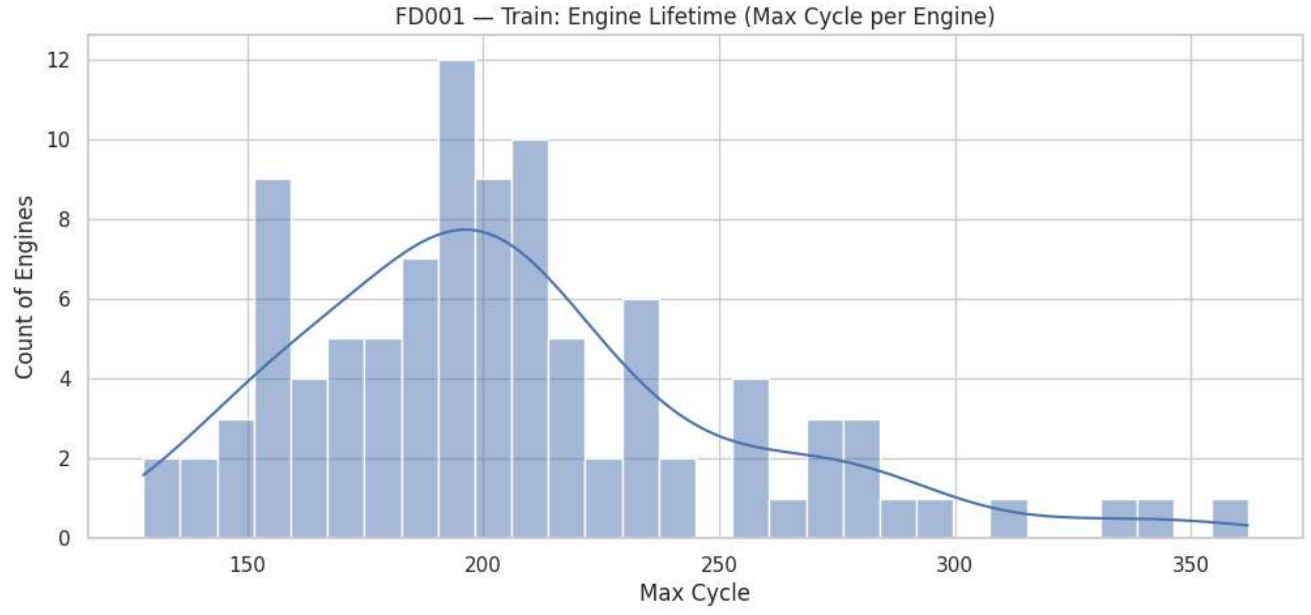
display(train_eng.describe().T)
display(test_eng.describe().T)
```

	count	mean	std	min	25%	50%	75%	max
unit_id	100.0	50.50	29.011492	1.0	25.75	50.5	75.25	100.0
max_cycle	100.0	206.31	46.342749	128.0	177.00	199.0	229.25	362.0
min_cycle	100.0	1.00	0.000000	1.0	1.00	1.0	1.00	1.0
n_rows	100.0	206.31	46.342749	128.0	177.00	199.0	229.25	362.0
start_rul	100.0	205.31	46.342749	127.0	176.00	198.0	228.25	361.0
end_rul	100.0	0.00	0.000000	0.0	0.00	0.0	0.00	0.0

	count	mean	std	min	25%	50%	75%	max
unit_id	100.0	50.500000	29.011492	1.0	25.75	50.5	75.25	100.0
max_cycle	100.0	130.960000	53.593479	31.0	88.75	133.5	164.25	303.0
min_cycle	100.0	1.000000	0.000000	1.0	1.00	1.0	1.00	1.0
n_rows	100.0	130.960000	53.593479	31.0	88.75	133.5	164.25	303.0
start_rul	99.0	204.434343	65.006618	56.0	150.50	204.0	253.50	381.0
end_rul	99.0	75.151515	41.813516	7.0	31.50	85.0	112.00	145.0

4) Engine Lifetime Distribution (Train)

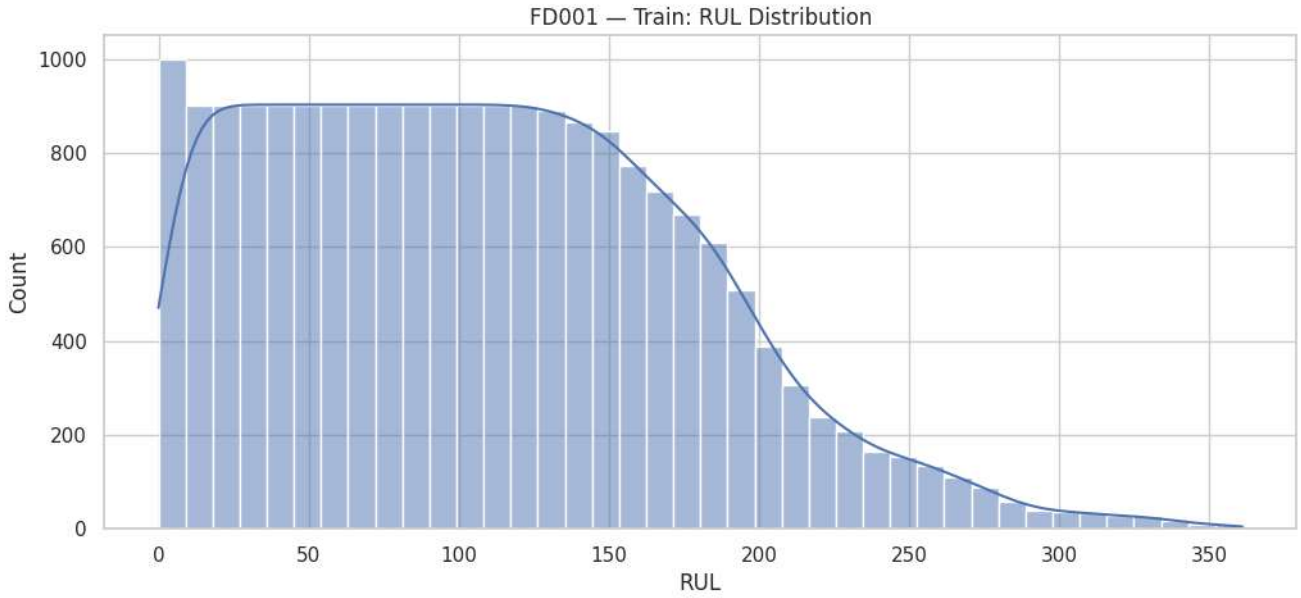
```
plt.figure()
sns.histplot(train_eng["max_cycle"], bins=30, kde=True)
plt.title(f"{FD} — Train: Engine Lifetime (Max Cycle per Engine)")
plt.xlabel("Max Cycle")
plt.ylabel("Count of Engines")
plt.show()
```

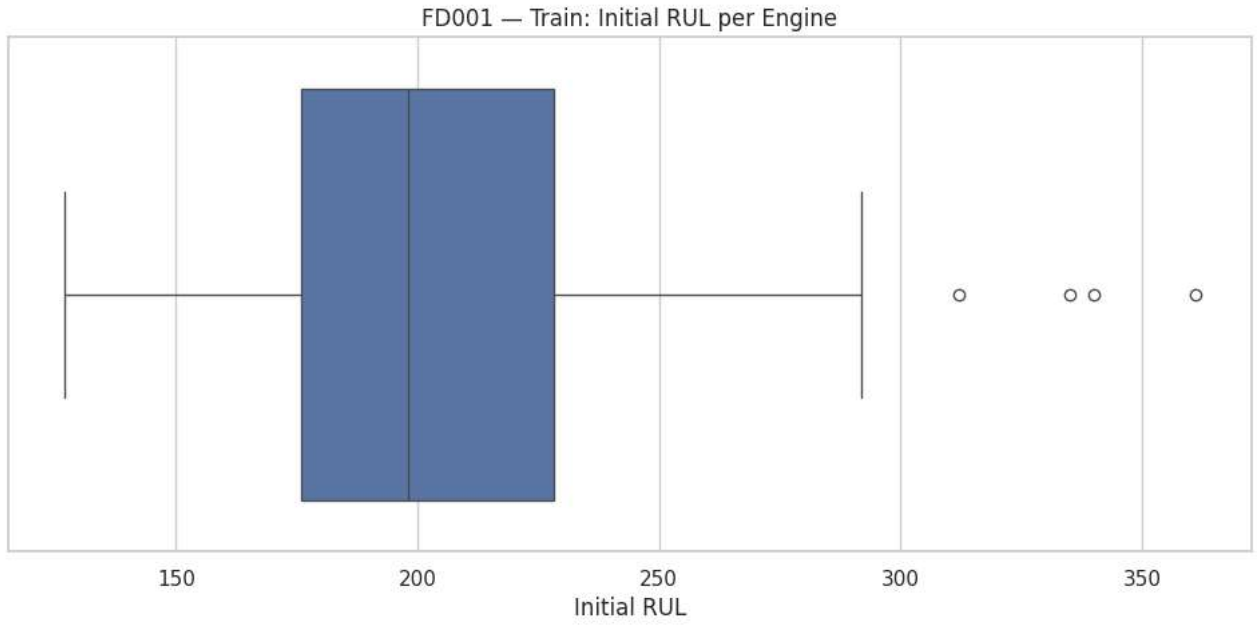


5) RUL Distribution (Train)

```
plt.figure()
sns.histplot(train["RUL"], bins=40, kde=True)
plt.title(f"{FD} — Train: RUL Distribution")
plt.xlabel("RUL")
plt.ylabel("Count")
plt.show()

plt.figure()
sns.boxplot(x=train_eng["start_rul"])
plt.title(f"{FD} — Train: Initial RUL per Engine")
plt.xlabel("Initial RUL")
plt.show()
```





6) Feature Groups (Sensors + Operating Settings)

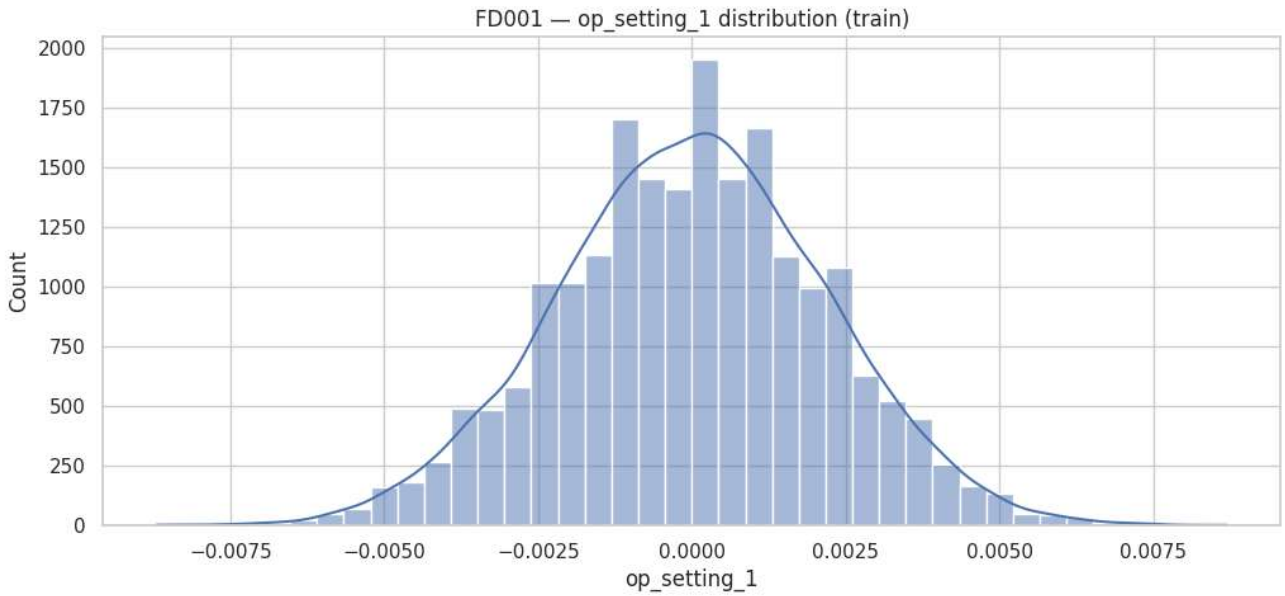
```
sensor_cols = [c for c in train.columns if c.startswith("s")]
op_cols = [c for c in train.columns if c.startswith("op_setting")]

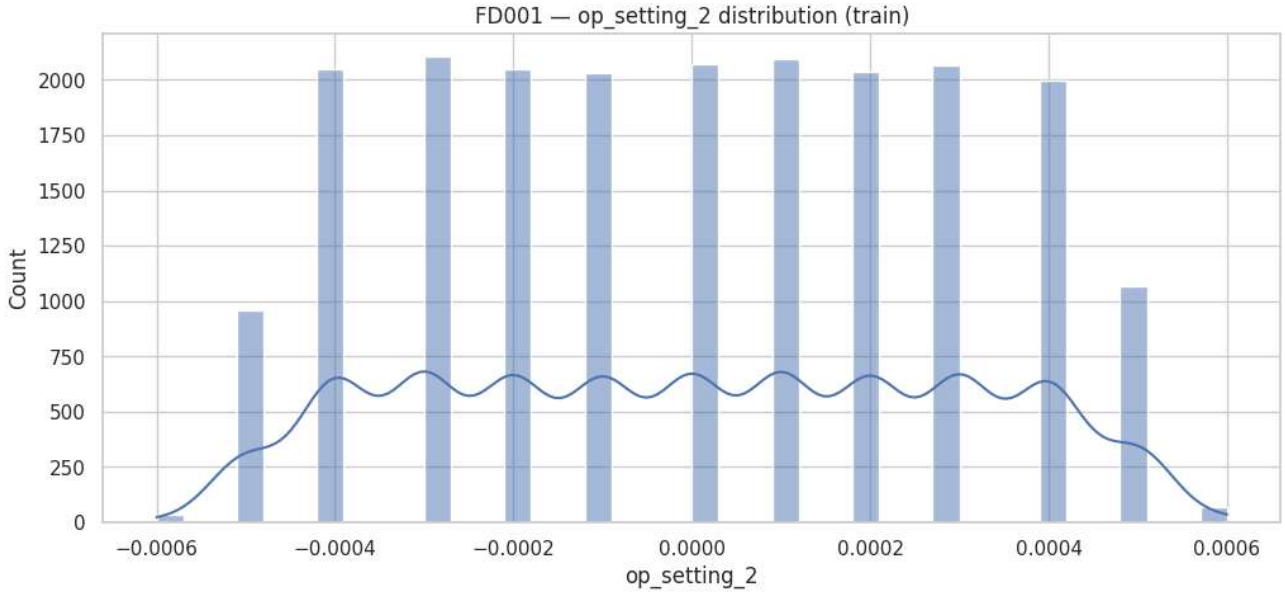
print("op cols:", op_cols)
print("sensor cols:", len(sensor_cols))
print("first sensors:", sensor_cols[:7])
```

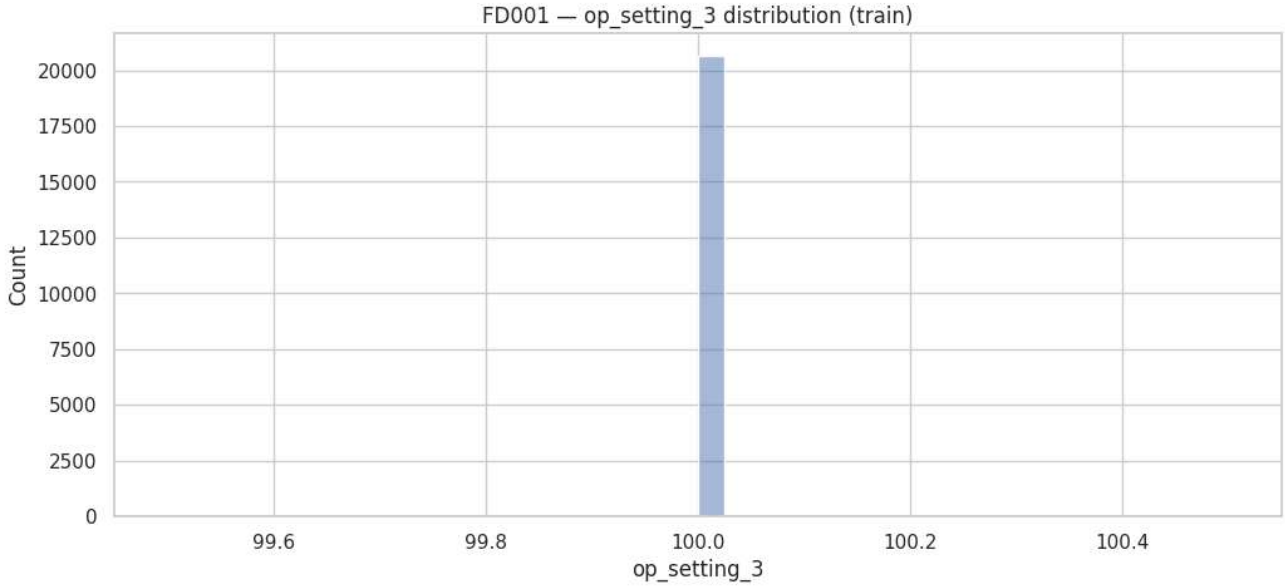
```
op cols: ['op_setting_1', 'op_setting_2', 'op_setting_3']
sensor cols: 21
first sensors: ['s1', 's2', 's3', 's4', 's5', 's6', 's7']
```

7) Operating Settings Distributions

```
for c in op_cols:
    plt.figure()
    sns.histplot(train[c], bins=40, kde=True)
    plt.title(f"{FD} — {c} distribution (train)")
    plt.xlabel(c)
    plt.ylabel("Count")
    plt.show()
```


FD001 — op_setting_1 distribution (train)
Count
op_setting_1
2000
1750
1500
1250
1000
750
500
250
0
−0.0075
−0.0050
−0.0025
0.0000
0.0025
0.0050
0.0075


FD001 — op_setting_2 distribution (train)
Count
op_setting_2
2000
1750
1500
1250
1000
750
500
250
0
−0.0006
−0.0004
−0.0002
0.0000
0.0002
0.0004
0.0006

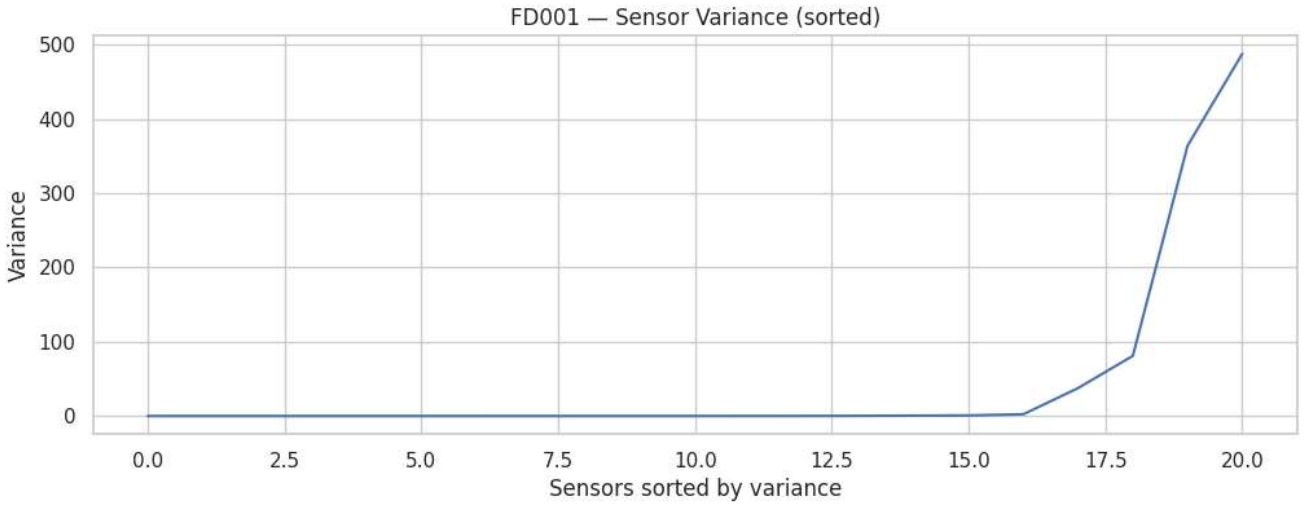

FD001 — op_setting_3 distribution (train)
Count
op_setting_3
20000
17500
15000
12500
10000
7500
5000
2500
0
99.6
99.8
100.0
100.2
100.4

8) Low-Variance Sensors (Variance Scan)

```
sensor_var = train[sensor_cols].var().sort_values()

plt.figure(figsize=(12,4))
sns.lineplot(x=np.arange(len(sensor_var)), y=sensor_var.values)
plt.title(f"{FD} — Sensor Variance (sorted)")
plt.xlabel("Sensors sorted by variance")
plt.ylabel("Variance")
plt.show()

VAR_THRESHOLD = 1e-8
low_var_sensors = sensor_var[sensor_var <= VAR_THRESHOLD].index.tolist()
print("Low-variance sensors:", low_var_sensors)
```




```
Low-variance sensors: ['s19', 's18', 's16', 's10', 's5', 's1']
```

9) Correlation with RUL (Top Signals)

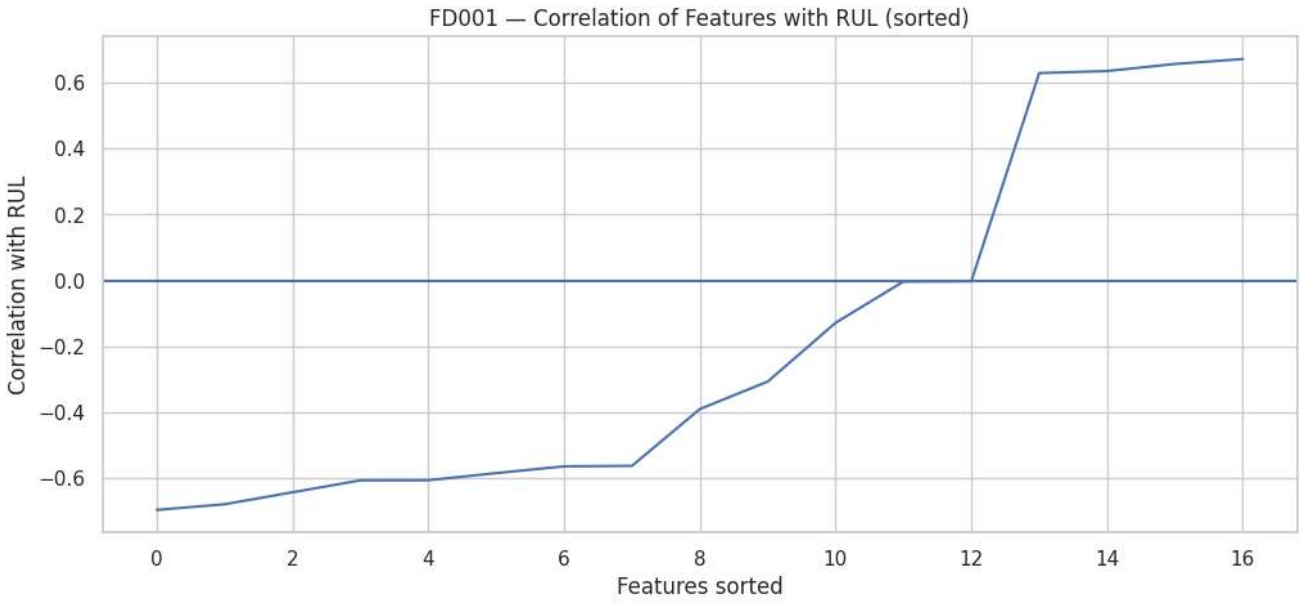
```
corr = train[sensor_cols + op_cols + ["RUL"]].corr(numeric_only=True)["RUL"].drop("RUL").sort_values()

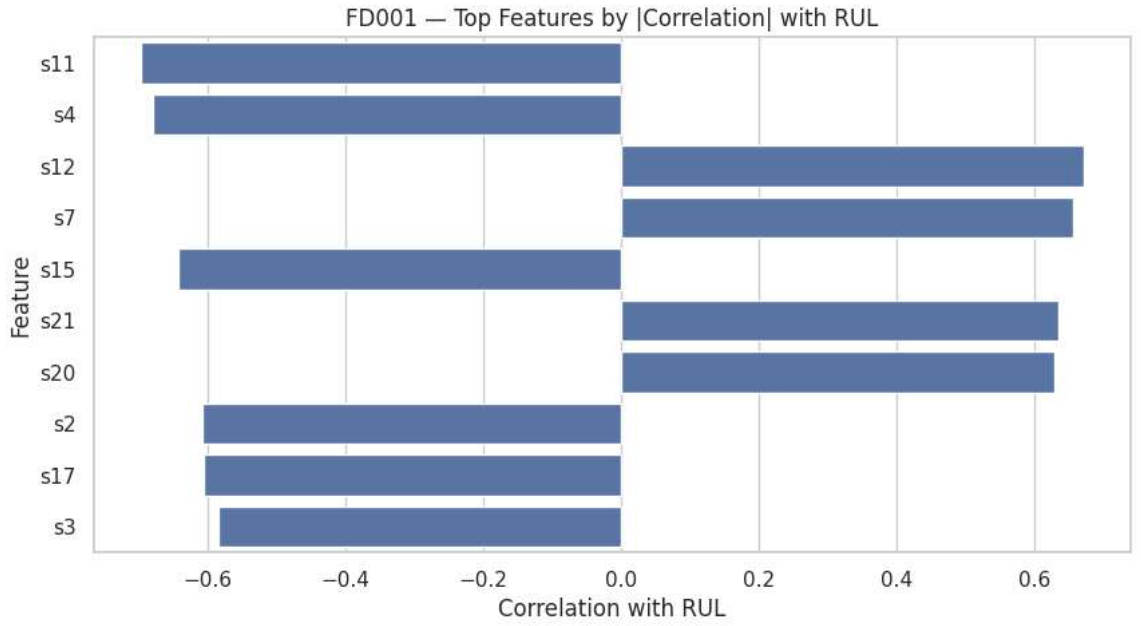
top_features = corr.abs().sort_values(ascending=False).head(10).index.tolist()
print("Top features by |corr|:", top_features)

plt.figure(figsize=(12,5))
sns.lineplot(x=np.arange(len(corr)), y=corr.values)
plt.axhline(0)
plt.title(f"{FD} — Correlation of Features with RUL (sorted)")
plt.xlabel("Features sorted")
plt.ylabel("Correlation with RUL")
plt.show()

plt.figure(figsize=(10,5))
sns.barplot(x=corr.loc[top_features].values, y=top_features, orient="h")
plt.title(f"{FD} — Top Features by |Correlation| with RUL")
plt.xlabel("Correlation with RUL")
plt.ylabel("Feature")
plt.show()
```

```
Top features by |corr|: ['s11', 's4', 's12', 's7', 's15', 's21', 's20', 's2', 's17', 's3']
```

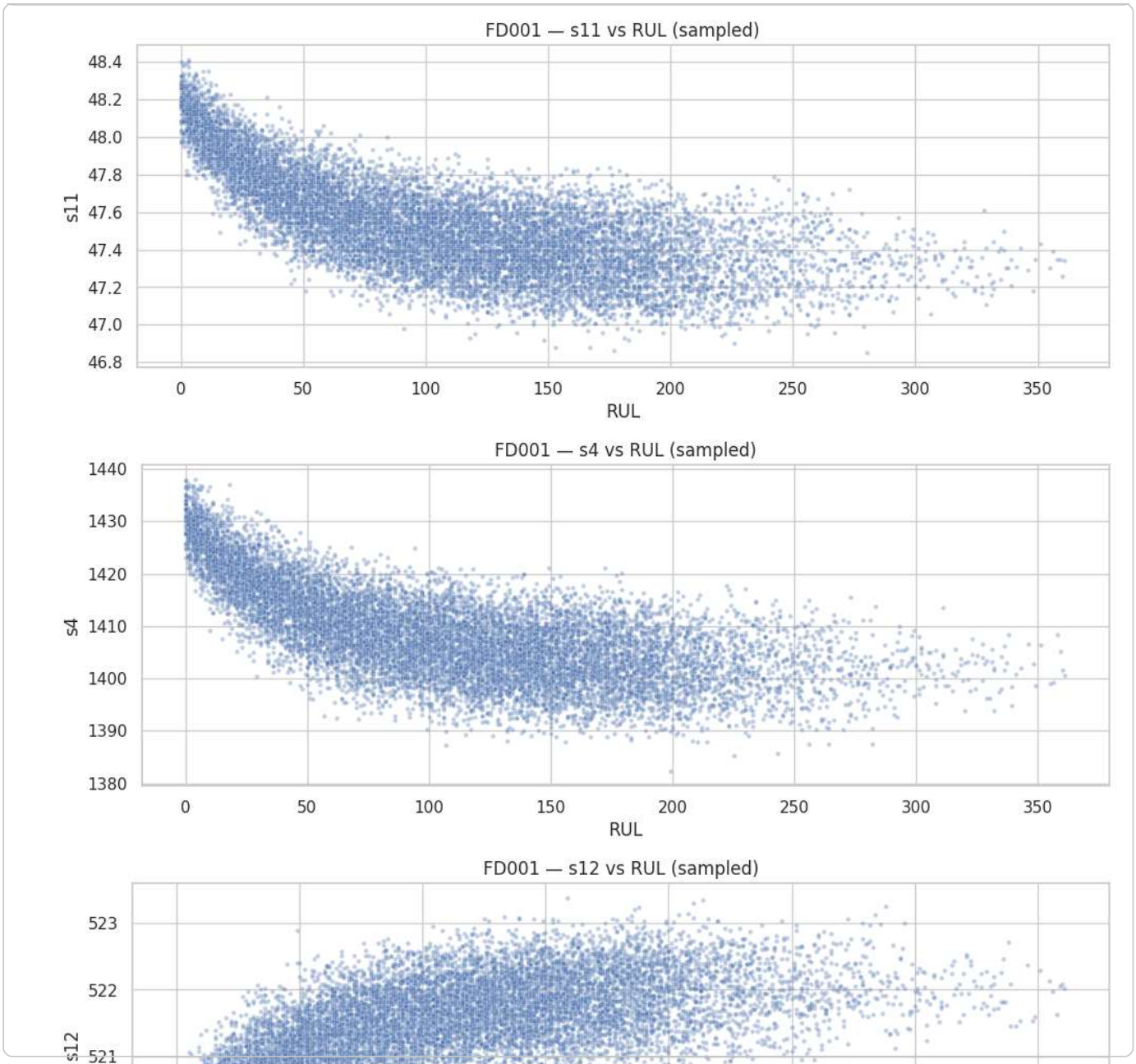




10) Feature vs RUL (Scatter using Sampling)

```
SAMPLE_N = min(12000, len(train))
sample = train[["RUL"] + top_features].sample(SAMPLE_N, random_state=42)

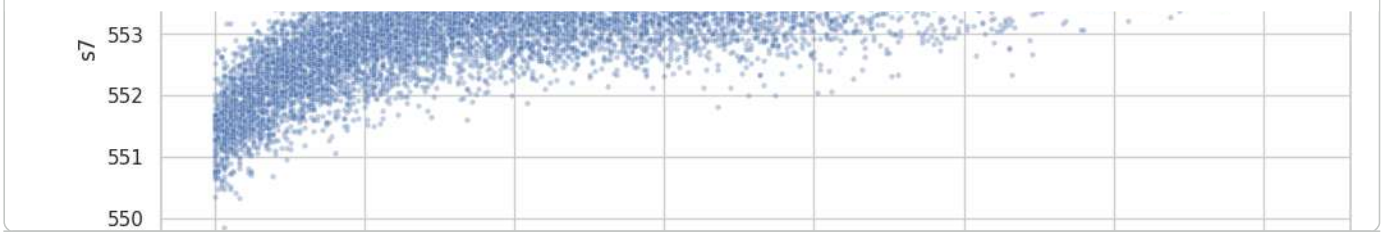
for feat in top_features[:6]:
    plt.figure(figsize=(12,4))
    sns.scatterplot(data=sample, x="RUL", y=feat, s=10, alpha=0.35)
    plt.title(f"{FD} — {feat} vs RUL (sampled)")
    plt.xlabel("RUL")
    plt.ylabel(feat)
    plt.show()
```

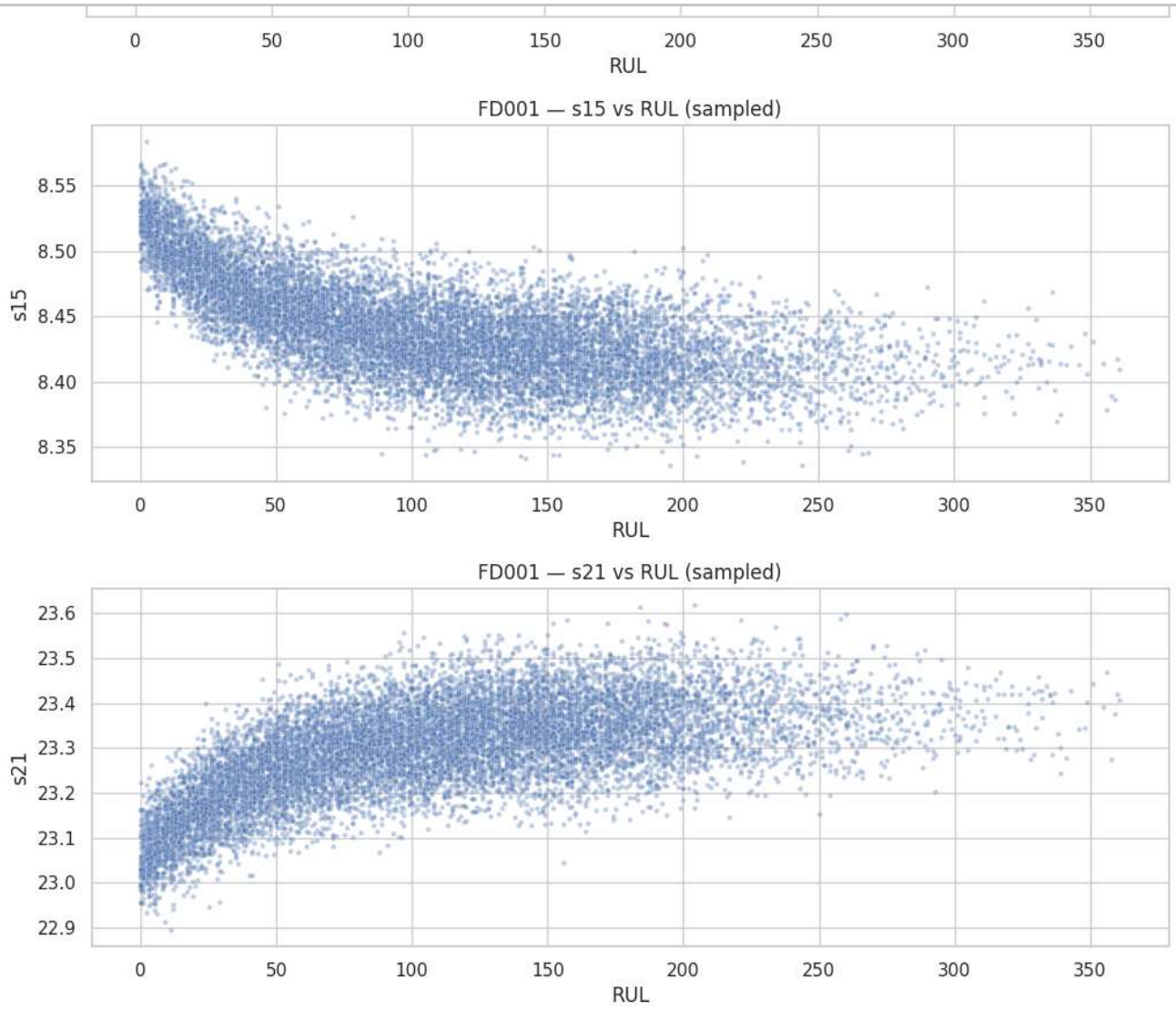


11) Feature Trend over RUL Bins (Mean Degradation)

```
BIN = 10
tmp = train[["RUL"] + top_features[:6]].copy()
tmp["RUL_bin"] = (tmp["RUL"] // BIN) * BIN
agg = tmp.groupby("RUL_bin")[top_features[:6]].mean().reset_index()

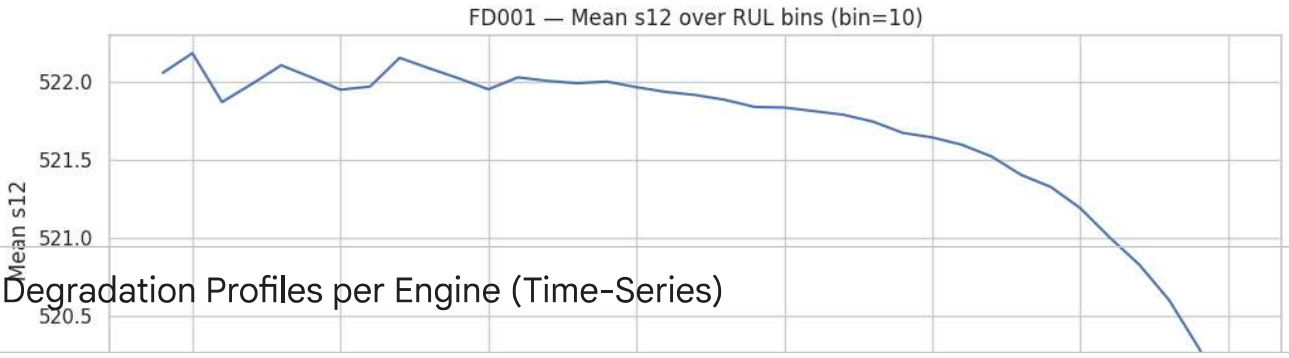
for feat in top_features[:6]:
    plt.figure(figsize=(12,4))
    sns.lineplot(data=agg, x="RUL_bin", y=feat)
    plt.gca().invert_xaxis()
    plt.title(f"{FD} — Mean {feat} over RUL bins (bin={BIN})")
    plt.xlabel("RUL (binned)")
    plt.ylabel(f"Mean {feat}")
    plt.show()
```




0
50
100
150
200
250
300
350
RUL
FD001 — s15 vs RUL (sampled)
8.55
8.50
8.45
8.40
8.35
s15
0
50
100
150
200
250
300
350
RUL
FD001 — s21 vs RUL (sampled)
23.6
23.5
23.4
23.3
23.2
23.1
23.0
22.9
s21
0
50
100
150
200
250
300
350
RUL





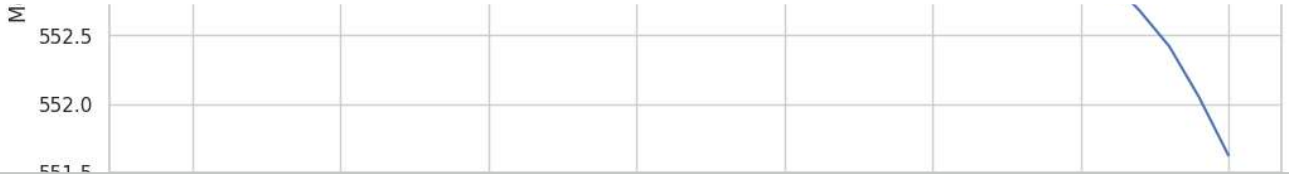


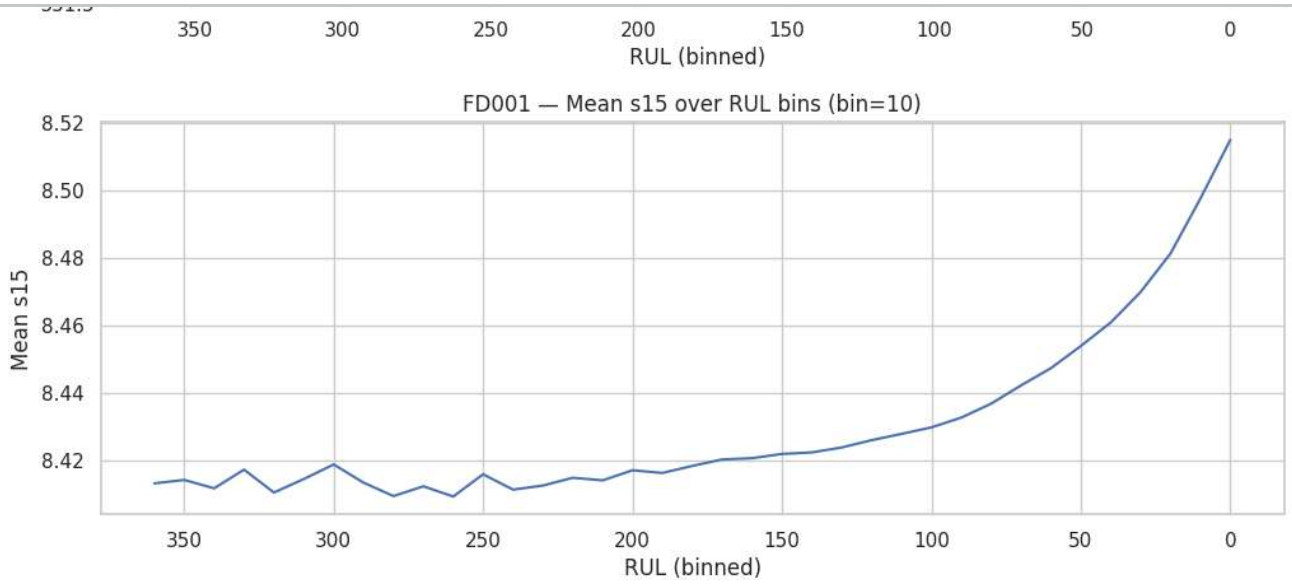
12) Degradation Profiles per Engine (Time-Series)

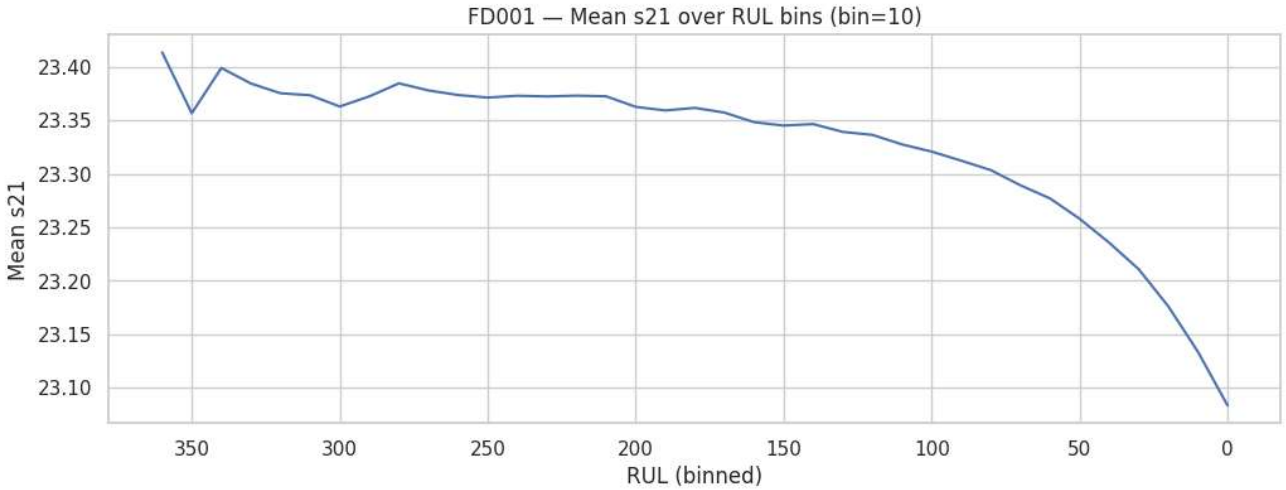
```
np.random.seed(42)
sample_engines = np.random.choice(train["unit_id"].unique(), size=3, replace=False)
print("Sample engines:", sample_engines)

plot_feats = top_features[:4]

for eng in sample_engines:
    df_e = train[train["unit_id"] == eng].sort_values("cycle")
    for feat in plot_feats:
        plt.figure(figsize=(12,4))
        sns.lineplot(data=df_e, x="cycle", y=feat)
        plt.title(f"{FD} — Engine {eng}: {feat} over cycles")
        plt.xlabel("Cycle")
        plt.ylabel(feat)
        plt.show()
```




350
300
250
200
150
100
50
0
RUL (binned)
FD001 — Mean s15 over RUL bins (bin=10)
8.52
8.50
8.48
8.46
8.44
8.42
Mean s15
350
300
250
200
150
100
50
0
RUL (binned)


FD001 — Mean s21 over RUL bins (bin=10)
23.40
23.35
23.30
23.25
23.20
23.15
23.10
Mean s21
350
300
250
200
150
100
50
0
RUL (binned)